TRƯỜNG ĐẠI HỌC VĂN LANG · KHOA CÔNG NGHỆ THÔNG TIN

CHUYÊN ĐỀ TỐT NGHIỆP 1 · HỌC KỲ 1, NĂM HỌC 2026 – 2027

Tài liệu tự học buổi 15 — Bảo mật, quyền truy cập và tài liệu kỹ thuật theo chuyên ngành

HTTPS, CORS, JWT, kiểm tra dữ liệu vào và API docs [SE] · Kiểm soát truy cập và ẩn danh hoá dữ liệu nhạy cảm [DA] · Model card, báo cáo thiên lệch và checklist AI có trách nhiệm [AI]. Mỗi phần có mẫu tài liệu điền sẵn và checklist bảo mật tự đánh giá.

Sản phẩm của tài liệu này là thành phần 5 và 6 của Bài tập 3 — checklist bảo mật đã tự đánh giá và tài liệu kỹ thuật theo chuyên ngành. Hoàn thiện ở workshop buổi 16, nộp cuối buổi 16.

Biên soạn: **TS. Nguyễn Trí Hải** · Bộ môn Kỹ thuật phần mềm

Tài liệu lưu hành nội bộ · Bộ tài liệu tự học phần chuyên ngành (30% thời lượng lý thuyết buổi 15) · Nội dung pháp lý cập nhật tới 10/2026, cần kiểm tra lại mỗi học kỳ. Tài liệu song ngữ: bản tiếng Việt ở nửa đầu, bản tiếng Anh (English version) ở nửa sau.

Cách dùng tài liệu này

Giảng viên giảng trên lớp (buổi 15)	Sinh viên tự đọc trong tài liệu này
Ba mức tái lập và vì sao Bài tập 3 yêu cầu mức 3; docker-compose hoàn chỉnh có healthcheck và dữ liệu mẫu; phép thử máy trắng; biến môi trường và bốn nơi tuyệt đối không để secret; năm mục của Deployment Guide (tiếng Anh); OWASP Top 10:2025 ở mức nhận biết; bảy mục checklist bảo mật bắt buộc.	Cách hiện thực từng mục bảo mật trên stack của track mình, và mẫu tài liệu kỹ thuật của track: API docs [SE] · Pipeline doc và Data dictionary [DA] · Model card [AI].

Bảy mục checklist bảo mật bắt buộc — nhắc lại từ bài giảng

1. Không có secret nào trong repo (mật khẩu, khoá API, chuỗi kết nối, token).
2. Dữ liệu vào được kiểm tra ở tầng ứng dụng, không chỉ dựa vào ràng buộc cơ sở dữ liệu.
3. Kiểm soát truy cập: người dùng chỉ đọc và sửa được dữ liệu thuộc quyền của mình.
4. Mật khẩu và dữ liệu nhạy cảm được băm hoặc mã hoá, không lưu ở dạng đọc được.
5. Thông báo lỗi không để lộ cấu trúc bên trong (vết ngăn xếp, tên bảng, chuỗi kết nối).
6. Nhật ký hệ thống không ghi mật khẩu, token, hay dữ liệu cá nhân không cần thiết.
7. Phụ thuộc bên ngoài được ghim phiên bản và đã kiểm cảnh báo lỗ hổng ở mức cơ bản.

Cách điền: mỗi mục ghi ĐÃ LÀM / CHƯA LÀM / KHÔNG ÁP DỤNG, kèm một dòng minh chứng (đường dẫn tệp, lệnh đã chạy kèm kết quả, **ảnh chụp kết quả chạy**, hoặc lý do nếu là "không áp dụng"). Ghi "chưa làm" trung thực KHÔNG bị trừ điểm; ghi "đã làm" mà không có minh chứng thì bị trừ.

Repo nộp BT3 phải PUBLIC (giảng viên không nhận lời mời collaborator). Mọi thứ trong lịch sử Git vì vậy là công khai — mục 1 của checklist là mục nghiêm trọng nhất.

Ranh giới phạm vi — đọc để không làm thừa

Nội dung	Thuộc học phần nào
Bảy mục checklist ở trên, tự đánh giá và ghi minh chứng	Chuyên đề tốt nghiệp 1 — bắt buộc ở Bài tập 3
Nhận biết được mười nhóm rủi ro OWASP Top 10:2025 và tự kiểm được năm nhóm ưu tiên	Chuyên đề tốt nghiệp 1 — mức nhận biết
Rà soát bảo mật đầy đủ, kiểm thử xâm nhập, mô hình hoá mối đe doạ	Chuyên đề tốt nghiệp 2
Quản lý secret bằng hệ thống chuyên dụng (vault), luân chuyển khoá tự động	Chuyên đề tốt nghiệp 2
Vận hành hệ thống giám sát đầy đủ, ứng phó sự cố bảo mật, tuân thủ chuẩn quốc tế (thiết kế cảnh báo bốn thành phần thì thuộc học phần này — buổi 15)	Chuyên đề tốt nghiệp 2 và các học phần chuyên sâu

PHẦN A — Track SE: HTTPS, CORS, JWT, kiểm tra dữ liệu vào và API docs

A1. HTTPS ở mức phạm vi học phần

Prototype của sinh viên chạy ở localhost nên không dùng HTTPS thật. Điều sinh viên cần chứng minh ở Bài tập 3 là: sinh viên BIẾT chỗ nào cần HTTPS và sinh viên đã chuẩn bị cho nó, chứ không phải sinh viên đã cài chứng chỉ.

Việc	Làm ở mức Bài tập 3	Không yêu cầu ở học phần này
Chuyển hướng sang HTTPS	Ghi trong Deployment Guide: "khi triển khai thật, đặt sau một máy chủ proxy có chứng chỉ TLS; ứng dụng đọc header X-Forwarded-Proto để biết yêu cầu gốc là HTTPS".	Cài đặt và cấu hình chứng chỉ TLS thật
Cookie an toàn	Nếu dùng cookie: đặt HttpOnly và SameSite ngay từ bây giờ; đặt Secure có điều kiện theo biến môi trường để localhost vẫn chạy được.	Cấu hình HSTS, ghim chứng chỉ
Không gửi secret qua URL	Kiểm ngay: token, mật khẩu, khoá không bao giờ nằm trong query string, vì query string bị ghi vào nhật ký máy chủ và lịch sử trình duyệt.	Không có

Cấu hình cookie an toàn có điều kiện — chạy được cả ở localhost

```
const LA_MOI_TRUONG_THAT = process.env.NODE_ENV === 'production';

res.cookie('refresh_token', token, {
  httpOnly: true,                    // mã JavaScript KHÔNG đọc được
  secure:   LA_MOI_TRUONG_THAT,      // chỉ gửi qua HTTPS khi là môi trường thật
  sameSite: LA_MOI_TRUONG_THAT ? 'strict' : 'lax',
  maxAge:   7 * 24 * 3600 * 1000,
  path:     '/api/auth',             // giới hạn đường dẫn cookie được gửi kèm
});

// Trong Deployment Guide, ghi rõ:
//   "Đặt NODE_ENV=production khi triển khai thật. Khi đó cookie chỉ được gửi
//    qua HTTPS. Nếu triển khai sau proxy, bật app.set('trust proxy', 1)
//    để Express đọc đúng X-Forwarded-Proto."
```

A2. CORS — cấu hình đúng, không mở cho mọi nguồn

Cấu hình CORS đúng cho Bài tập 3

```
const cors = require('cors');

// SAI - mở cho mọi nguồn, KHÔNG được dùng khi có xác thực bằng cookie
// app.use(cors());
// app.use(cors({ origin: '*', credentials: true }));   // trình duyệt cũng từ chối

// ĐÚNG - danh sách nguồn cho phép, lấy từ biến môi trường
const NGUON_CHO_PHEP = (process.env.CORS_ORIGINS || 'http://localhost:5173')
  .split(',').map(s => s.trim()).filter(Boolean);

app.use(cors({
  origin: (nguon, cb) => {
    // nguồn rỗng nghĩa là yêu cầu không từ trình duyệt (curl, ứng dụng di động)
    if (!nguon || NGUON_CHO_PHEP.includes(nguon)) return cb(null, true);
    cb(new Error('Nguồn không được phép bởi cấu hình CORS'));
  },
  credentials: true,
  methods: ['GET', 'POST', 'PATCH', 'DELETE'],
  allowedHeaders: ['Content-Type', 'Authorization', 'X-Request-Id'],
  maxAge: 600,
}));

// .env.example:
//   CORS_ORIGINS=http://localhost:5173,http://localhost:3000
```

Ba điều cần hiểu về CORS để trả lời được câu hỏi ở buổi 16

CORS là cơ chế của TRÌNH DUYỆT, không phải cơ chế bảo mật của máy chủ. Nó ngăn trang web A gọi API của sinh viên bằng thông tin đăng nhập của người dùng. Nó KHÔNG ngăn được ai gọi API bằng curl. Vì vậy CORS không thay thế cho xác thực và kiểm soát truy cập.

origin: "*" cùng với credentials: true là cấu hình không hợp lệ. Trình duyệt sẽ từ chối. Nếu sinh viên mở cho mọi nguồn thì phải bỏ credentials, và khi đó cookie không được gửi.

Danh sách nguồn cho phép lấy từ biến môi trường. Ghi cứng địa chỉ frontend trong mã nguồn là lỗi cấu hình: khi triển khai lên tên miền khác, sinh viên phải sửa mã và dựng lại.

A3. JWT — bốn lỗi phổ biến và cách tránh

Lỗi	Vì sao nguy hiểm	Cách làm đúng
Khoá bí mật ghi trong mã nguồn hoặc quá ngắn	Ai có khoá thì tạo được token hợp lệ cho bất kỳ người dùng nào, kể cả quản trị viên	Sinh khoá ngẫu nhiên ít nhất 32 byte, đặt trong biến môi trường; .env.example chỉ ghi giá trị mẫu
Không kiểm hạn dùng, hoặc hạn quá dài	Token bị lộ vẫn dùng được vô thời hạn; không thể vô hiệu hoá	Token truy cập hạn ngắn (15–60 phút); nếu cần lâu hơn thì dùng thêm token làm mới lưu trong cookie HttpOnly
Tin vào nội dung token mà không xác thực chữ ký, hoặc chấp nhận thuật toán "none"	Ai cũng sửa được vai trò trong token thành admin	Luôn dùng hàm verify với danh sách thuật toán chỉ định rõ; không dùng hàm decode để ra quyết định phân quyền
Đưa dữ liệu cá nhân vào phần payload của token	Payload của JWT chỉ được mã hoá base64, ai cũng đọc được nội dung	Payload chỉ chứa id người dùng, vai trò và thời hạn. Không đưa email, số điện thoại, địa chỉ vào token

src/middleware/auth.js — xác thực token đúng cách

```
const jwt = require('jsonwebtoken');

const KHOA = process.env.JWT_SECRET;
if (!KHOA || KHOA.length < 32) {
  // Dừng ngay khi khởi động, không để hệ thống chạy với cấu hình không an toàn
  throw new Error('JWT_SECRET phải có ít nhất 32 ký tự');
}

function xacThuc(req, res, next) {
  const header = req.headers.authorization || '';
  const token = header.startsWith('Bearer ') ? header.slice(7) : null;
  if (!token) return res.status(401).json({ error: 'CHUA_DANG_NHAP' });
  try {
    // CHỈ ĐỊNH RÕ thuật toán - không để thư viện tự chọn theo header của token
    const payload = jwt.verify(token, KHOA, { algorithms: ['HS256'] });
    req.user = { id: payload.sub, vai_tro: payload.vai_tro };
    next();
  } catch (e) {
    // KHÔNG trả về chi tiết lỗi của thư viện cho người gọi
    return res.status(401).json({ error: 'TOKEN_KHONG_HOP_LE' });
  }
}

// Kiểm soát truy cập theo CHỦ SỞ HỮU - mục số 3 của checklist,
// và là lỗ hổng phổ biến nhất trong prototype sinh viên (A01 OWASP)
function chiChuSoHuu(layChuSoHuu) {
  return async (req, res, next) => {
    const chuSoHuuId = await layChuSoHuu(req);
    if (chuSoHuuId === null || chuSoHuuId === undefined)
      return res.status(404).json({ error: 'KHONG_TIM_THAY' });
    // So sánh cùng kiểu: id trong token có thể là chuỗi, id trong CSDL là số
    if (String(chuSoHuuId) !== String(req.user.id) && req.user.vai_tro !== 'QUAN_TRI')
      return res.status(403).json({ error: 'KHONG_CO_QUYEN' });
    next();
  };
}

module.exports = { xacThuc, chiChuSoHuu };
```

A4. Kiểm tra dữ liệu vào — chống injection ở mức thực hành

Loại	Cách hệ thống bị tấn công	Cách phòng ở Bài tập 3	Cách sinh viên tự kiểm
SQL injection	Ghép chuỗi truy vấn với dữ liệu người dùng	Luôn dùng tham số hoá ($1, ?) hoặc ORM. TUYỆT ĐỐI không ghép chuỗi SQL.	grep tìm dấu ghép chuỗi trong các câu truy vấn; nhập chuỗi ' OR 1=1 -- vào ô tìm kiếm
Cross-site scripting	Dữ liệu người dùng được chèn vào HTML và bị thực thi như mã	Để framework tự thoát ký tự (React và Vue làm sẵn); không dùng innerHTML hay dangerouslySetInnerHTML với dữ liệu người dùng	grep tìm innerHTML và dangerouslySetInnerHTML; nhập một chuỗi thẻ script vào một trường văn bản rồi xem nó hiển thị hay bị thực thi
Command injection	Dữ liệu người dùng được đưa vào một lệnh hệ thống	Không gọi lệnh hệ thống với dữ liệu người dùng. Nếu buộc phải, dùng dạng truyền mảng tham số chứ không truyền chuỗi shell.	grep tìm exec, spawn, os.system, subprocess với shell=True
Vượt đường dẫn	Người dùng gửi ../../etc/passwd làm tên tệp	Không nhận đường dẫn từ người dùng. Nếu cần tải tệp, dùng id chứ không dùng tên tệp.	Thử gọi endpoint tải tệp với tên tệp chứa ../

Ba lệnh grep chạy trong ba phút, phát hiện phần lớn vấn đề

1. grep -rnE '(SELECT|INSERT|UPDATE|DELETE).*(\+ *[a-zA-Z_]|\$\{)' src/ — tìm câu SQL bị ghép chuỗi hoặc chèn biến bằng template string.

2. grep -rnE "innerHTML|dangerouslySetInnerHTML|v-html" src/ — tìm chỗ chèn HTML thô.

3. grep -rnE 'exec\(|execSync|spawn\(|os\.system|shell=True' src/ — tìm chỗ gọi lệnh hệ thống.

Chụp ảnh kết quả ba lệnh và ghi vào mục 2 của checklist bảo mật làm minh chứng. Nếu kết quả trống, đó là minh chứng tốt; nếu có kết quả, xem từng chỗ và sửa hoặc giải thích vì sao chỗ đó an toàn.

A5. API docs — thành phần 6 của Bài tập 3 cho track SE

API docs phải mô tả đúng thứ hệ thống THẬT SỰ làm. Tài liệu mô tả một endpoint chưa tồn tại là lỗi nhất quán và bị trừ điểm ở tiêu chí 3 của rubric.

Cách làm	Ưu điểm	Yêu cầu để đạt Bài tập 3
OpenAPI / Swagger	Sinh được trang tài liệu tương tác; có thể sinh tự động từ mã nguồn với một số framework	Có tệp openapi.yaml hoặc openapi.json trong repo; mở được bằng Swagger UI hoặc dán vào editor.swagger.io mà không lỗi
Postman collection	Dễ làm nhất; đồng thời là bộ kiểm thử API thủ công	Xuất tệp .postman_collection.json và commit vào repo; mỗi request có ví dụ phản hồi đã lưu từ lần chạy thật
Tệp Markdown viết tay	Không cần công cụ; đọc được ngay trên GitHub	Mỗi endpoint đủ bảy mục ở bảng dưới; ít nhất một ví dụ request và response THẬT cho mỗi endpoint

Bảy mục bắt buộc cho MỖI endpoint	Ví dụ
1. Phương thức và đường dẫn	POST /api/tickets
2. Mô tả một câu về nghiệp vụ	Tiếp nhận một thiết bị vào bảo hành, tạo phiếu và tính hạn xử lý theo mức ưu tiên.
3. Yêu cầu xác thực và quyền	Bắt buộc có token; vai trò NHAN_VIEN_TIEP_NHAN hoặc QUAN_TRI.
4. Tham số và thân yêu cầu	Bảng liệt kê từng trường: tên, kiểu, bắt buộc hay không, ràng buộc. Kèm một ví dụ JSON thật.
5. Thân phản hồi khi thành công	Mã 201 kèm ví dụ JSON THẬT chép từ một lần chạy, không phải ví dụ tự nghĩ ra.
6. Bảng mã lỗi có thể trả về	400 thiếu trường bắt buộc · 401 chưa đăng nhập · 403 không có quyền · 404 khách hàng không tồn tại · 409 thiết bị đang có phiếu mở · 422 kỹ thuật viên quá tải.
7. Ghi chú về giới hạn hoặc hành vi đặc biệt	Nếu gọi lại với cùng serial trong khi phiếu trước còn mở thì trả 409 chứ không tạo phiếu thứ hai.

A6. Checklist tự kiểm [SE]

☐ Đã chạy ba lệnh grep ở mục A4 và ghi kết quả vào checklist bảo mật.

- ☐ JWT_SECRET lấy từ biến môi trường, dài ít nhất 32 ký tự, và hệ thống dừng khi thiếu.
- ☐ Hàm xác thực token chỉ định rõ thuật toán, không dùng decode để phân quyền.
- ☐ Payload token chỉ chứa id, vai trò và thời hạn — không chứa dữ liệu cá nhân.
- ☐ Có kiểm soát truy cập theo chủ sở hữu, và đã tự kiểm bằng test case như mục A01 OWASP.
- ☐ CORS chỉ mở cho các nguồn trong biến môi trường, không dùng origin: "*" cùng credentials.
- ☐ Cookie (nếu dùng) có HttpOnly và SameSite; Secure đặt theo biến môi trường.
- ☐ Không có secret, token hay mật khẩu nào trong URL.
- ☐ Thông báo lỗi không chứa vết ngăn xếp, tên bảng hay chuỗi kết nối.
- ☐ API docs mô tả đủ mọi endpoint đang có, mỗi endpoint đủ bảy mục và có ví dụ THẬT.
- ☐ Deployment Guide có mục nêu cách bật HTTPS khi triển khai thật.

PHẦN B — Track DA: Kiểm soát truy cập và ẩn danh hoá dữ liệu nhạy cảm

B1. Phân loại dữ liệu — làm trước khi làm bất cứ gì khác

Không thể bảo vệ dữ liệu nếu chưa biết dữ liệu nào cần bảo vệ. Bước đầu tiên là đi qua từng cột trong kho dữ liệu của sinh viên và gán một trong bốn mức. Việc này mất khoảng ba mươi phút và là nền cho mọi quyết định sau đó.

Mức	Định nghĩa	Ví dụ trong bộ dữ liệu Mekong Mobile	Cách xử lý
CÔNG KHAI	Công bố ra ngoài cũng không gây hại	product_name, thuong_hieu, nhom_san_pham, store_name, thanh_pho	Không cần xử lý
NỘI BỘ	Chỉ dùng trong tổ chức, không định danh cá nhân	so_gio_xu_ly, tre_han, chi_phi_linh_kien, category_name, số liệu tổng hợp	Kiểm soát truy cập theo vai trò
ĐỊNH DANH CÁ NHÂN	Xác định được một người cụ thể	ho_ten, so_dien_thoai, email, dia_chi, serial_no (gắn với một thiết bị của một người)	Ẩn danh hoá hoặc giả danh hoá trong mọi bảng phân tích; chỉ giữ dạng gốc ở nơi nghiệp vụ thật sự cần
NHẠY CẢM	Rò rỉ gây hậu quả nghiêm trọng	Trong dữ liệu này không có; nếu dự án của sinh viên có số căn cước, dữ liệu sức khoẻ, dữ liệu tài chính cá nhân thì thuộc mức này	Mã hoá khi lưu; giới hạn truy cập tối đa; ghi nhật ký mọi lần truy cập

Nguyên tắc TỐI THIỂU HOÁ áp dụng vào kho dữ liệu

Kho dữ liệu phân tích thường KHÔNG cần tên, số điện thoại và địa chỉ đầy đủ. Nó cần biết "đây là cùng một khách hàng" và "khách này ở tỉnh nào".

Vì vậy: thay ho_ten và so_dien_thoai bằng một mã giả danh ổn định; giữ tinh_thanh nhưng bỏ số nhà và tên đường; giữ nhóm tuổi thay vì ngày sinh.

Phép thử: với mỗi cột định danh cá nhân trong kho dữ liệu, hỏi "báo cáo nào cần chính xác cột này ở dạng gốc?". Nếu không có báo cáo nào, cột đó không nên nằm trong kho ở dạng gốc.

Ghi kết quả vào Data dictionary: mỗi cột có thêm hai trường là mức phân loại và cách xử lý. Đây là phần đáng giá nhất trong Data dictionary của sinh viên và cũng là điều phân biệt bài điểm cao.

B2. Ba kỹ thuật ẩn danh hoá — chọn đúng kỹ thuật cho đúng mục đích

Kỹ thuật	Cách làm	Còn nối được các bản ghi của cùng một người?	Dùng khi nào
Giả danh hoá (pseudonymisation)	Thay giá trị định danh bằng một mã ổn định, sinh bằng HMAC-SHA256 với một khoá bí mật lấy từ biến môi trường	CÓ — cùng một người luôn ra cùng một mã	Kho dữ liệu phân tích: cần nối lịch sử của một khách hàng nhưng không cần biết đó là ai

Kỹ thuật	Cách làm	Còn nối được các bản ghi của cùng một người?	Dùng khi nào
Che một phần (masking)	Giữ một phần giá trị, che phần còn lại: 0912***678	Không đáng tin — nhiều người có thể ra cùng một dạng che	Màn hình hỗ trợ khách hàng: nhân viên cần đối chiếu nhanh với khách qua điện thoại
Tổng hợp (aggregation)	Không lưu dòng cá nhân, chỉ lưu số liệu theo nhóm; và ẩn nhóm có quá ít bản ghi	KHÔNG	Báo cáo chia sẻ ra ngoài, hoặc dashboard cho người không có quyền xem dữ liệu cá nhân

src/transform/gia_danh.py — giả danh hoá TRONG PIPELINE, khoá không bao giờ vào CSDL

```
import hashlib, hmac, os

# Khoá bí mật lấy từ biến môi trường - KHÔNG ghi trong mã, KHÔNG lưu trong CSDL.
# Không băm "trần": số điện thoại chỉ có 10 chữ số, bảng băm không khoá
# bị đoán ngược dễ dàng bằng cách thử hết các số.
KHOA = os.environ["PSEUDO_KEY"].encode()      # .env.example: PSEUDO_KEY=doi-gia-tri-nay
if len(KHOA) < 32:
    raise SystemExit("PSEUDO_KEY phải có ít nhất 32 ký tự")

def gia_danh(gia_tri: str | None) -> str | None:
    """Cùng một số điện thoại luôn ra cùng một mã - vẫn nối được lịch sử khách."""
    if not gia_tri:
        return None
    return hmac.new(KHOA, gia_tri.encode(), hashlib.sha256).hexdigest()

# Trong bước transform:
#   df["ma_khach_gia_danh"] = df["sdt_chuan"].map(gia_danh)
```

sql/030_view_phan_tich.sql — view phân tích và view tổng hợp có ngưỡng nhóm

```
-- VIEW cho người làm phân tích: KHÔNG có tên, số điện thoại, địa chỉ đầy đủ
CREATE OR REPLACE VIEW v_khach_hang_phan_tich AS
SELECT
  dc.customer_sk,                      -- khoá thay thế, không mang nghĩa nghiệp vụ
  dc.ma_khach_gia_danh,                -- tính sẵn trong pipeline (gia_danh.py)
  dc.tinh_thanh,                       -- giữ tỉnh, BỎ số nhà và tên đường
  date_trunc('month', dc.ngay_tao)::DATE AS thang_tao,   -- giảm độ chi tiết thời gian
  dc.nhom_khach                        -- phân loại nghiệp vụ, không định danh
FROM dim_customer dc;

-- VIEW tổng hợp cho dashboard chia sẻ rộng: ẨN nhóm quá ít khách
CREATE OR REPLACE VIEW v_bao_cao_tinh AS
SELECT k.tinh_thanh,
       COUNT(DISTINCT k.customer_sk)                              AS so_khach,
       ROUND(COUNT(f.ticket_sk)::NUMERIC
             / COUNT(DISTINCT k.customer_sk), 2)                  AS so_phieu_tb
FROM      v_khach_hang_phan_tich k
LEFT JOIN fact_ticket f ON f.customer_sk = k.customer_sk
GROUP BY  k.tinh_thanh
HAVING    COUNT(DISTINCT k.customer_sk) >= 20;  -- NGƯỠNG NHÓM tối thiểu: tỉnh dưới
                                                -- 20 khách bị ẩn, vì nhóm quá nhỏ
                                                -- có thể suy ra từng cá nhân
```

Vì sao ngưỡng "nhóm tối thiểu" là mục quan trọng nhất của mục B2

Một báo cáo tổng hợp trông có vẻ an toàn vẫn có thể để lộ cá nhân: nếu một tỉnh chỉ có một khách hàng, thì "doanh thu trung bình của tỉnh X" chính là doanh thu của người đó.

Kết hợp nhiều báo cáo tổng hợp cũng suy ra được cá nhân: biết tổng của nhóm 5 người và tổng của nhóm 4 người trong đó thì biết được người thứ năm.

Cách xử lý ở mức Bài tập 3: đặt một ngưỡng nhóm tối thiểu (20 là con số hợp lý cho dữ liệu này), ẩn mọi nhóm dưới ngưỡng, và GHI RÕ ngưỡng đó trong Data dictionary cùng lý do.

Kỹ thuật đầy đủ (differential privacy, l-diversity) thuộc phạm vi các học phần chuyên sâu. Ở đây sinh viên chỉ cần nhận ra rủi ro và áp dụng biện pháp đơn giản đúng cách.

B3. Kiểm soát truy cập theo vai trò trong kho dữ liệu

sql/040_quyen_truy_cap.sql — ba vai trò, ba mức truy cập

```
-- Vai trò 1: người làm phân tích - chỉ đọc VIEW đã giả danh hoá
CREATE ROLE nha_phan_tich;
GRANT USAGE  ON SCHEMA public TO nha_phan_tich;
GRANT SELECT ON v_khach_hang_phan_tich, v_bao_cao_tinh, fact_ticket TO nha_phan_tich;
-- KHÔNG cấp quyền trên dim_customer (bảng chứa dữ liệu gốc). View vẫn đọc được
-- dim_customer vì view chạy với quyền của người tạo view.

-- Vai trò 2: nhân viên nghiệp vụ - chỉ thấy phiếu của trung tâm mình
CREATE ROLE nhan_vien_nghiep_vu;
GRANT USAGE  ON SCHEMA public TO nhan_vien_nghiep_vu;
GRANT SELECT ON dim_customer, fact_ticket TO nhan_vien_nghiep_vu;

-- Giới hạn theo DÒNG bằng Row Level Security.
-- Lưu ý: bật RLS thì vai trò nào KHÔNG có policy sẽ thấy 0 dòng,
-- nên phải khai báo policy cho CẢ HAI vai trò đọc fact_ticket.
ALTER TABLE fact_ticket ENABLE ROW LEVEL SECURITY;
CREATE POLICY chi_trung_tam_cua_minh ON fact_ticket FOR SELECT
  TO nhan_vien_nghiep_vu
  USING (center_sk = current_setting('app.center_sk')::INT);
CREATE POLICY phan_tich_doc_tat_ca ON fact_ticket FOR SELECT
  TO nha_phan_tich
  USING (true);

-- Vai trò 3: tài khoản của pipeline - quyền tối thiểu để NẠP dữ liệu.
-- Cần SELECT vì UPDATE ... WHERE và INSERT ... ON CONFLICT phải đọc khoá.
-- Không có DELETE, không có quyền tạo/xoá bảng; KHÔNG dùng cho dashboard.
CREATE ROLE pipeline_writer;
GRANT USAGE  ON SCHEMA public TO pipeline_writer;
GRANT SELECT, INSERT, UPDATE ON staging_customers, dim_customer, fact_ticket
  TO pipeline_writer;
CREATE POLICY pipeline_ghi ON fact_ticket FOR ALL TO pipeline_writer
  USING (true) WITH CHECK (true);

-- Nguyên tắc: dashboard kết nối bằng tài khoản thuộc vai trò nha_phan_tich,
-- KHÔNG bằng tài khoản chủ sở hữu cơ sở dữ liệu. Đây là mục số 3 của checklist.
```

Ba điều phải đúng về kết nối của dashboard

1. Dashboard KHÔNG kết nối bằng tài khoản chủ sở hữu cơ sở dữ liệu. Nếu dashboard có lỗ hổng thì kẻ tấn công có toàn quyền. Dùng một tài khoản riêng chỉ có quyền SELECT trên các view cần thiết.

2. Dashboard đọc VIEW đã giả danh hoá, không đọc bảng gốc. Nhờ vậy ngay cả người có quyền vào dashboard cũng không thấy được số điện thoại và tên khách hàng.

3. Thông báo lỗi của dashboard không được in ra chuỗi kết nối hay tên bảng. Đây là mục số 5 của checklist bảo mật và là lỗi rất hay gặp với Streamlit: hãy đặt client.showErrorDetails = false trong .streamlit/config.toml để trang không in vết ngăn xếp.

B4. Pipeline doc và Data dictionary — thành phần 6 của Bài tập 3

Pipeline doc — sáu mục

Mục	Nội dung tối thiểu
1. Sơ đồ các pha	Hình vẽ: nguồn → extract → transform → quality → load → đích. Ghi rõ tệp mã nào phụ trách pha nào.
2. Nguồn dữ liệu	Mỗi nguồn: tên, định dạng, tần suất cập nhật, ai là người sở hữu, đường dẫn hoặc cách truy cập.
3. Đích và grain	Mỗi bảng đích: grain phát biểu bằng một câu, khoá nghiệp vụ, khoá thay thế.
4. Quy tắc làm sạch	Bảng liệt kê mỗi quy tắc: mã quy tắc, làm gì, ngưỡng, số dòng bị loại ở lần chạy gần nhất.
5. Lịch chạy và cách chạy bù	Lịch dự kiến; lệnh chạy bù một ngày thất bại; điều gì xảy ra nếu chạy hai lần cùng một ngày.
6. Hạn chế đã biết và độ trễ dữ liệu	Những vấn đề của dữ liệu nguồn không sửa được và ảnh hưởng tới báo cáo nào; số liệu tính tới thời điểm nào.

Data dictionary — mẫu điền sẵn hai dòng, mỗi cột đủ sáu trường theo đề BT3 (kiểu · ý nghĩa · giá trị hợp lệ · rỗng · phân loại · xử lý bảo mật)

Bảng.cột	Kiểu	Ý nghĩa	Giá trị hợp lệ	Rỗng?	Phân loại	Xử lý bảo mật
dim_customer.sdt_chuan	VARCHAR(10)	Số điện thoại đã chuẩn hoá về 10 chữ số bắt đầu bằng 0. Là khoá nghiệp vụ để hợp nhất khách hàng trùng lặp.	Đúng 10 chữ số, bắt đầu bằng 0.	Có (khoảng 6,1% bản ghi nguồn thiếu số điện thoại)	ĐỊNH DANH CÁ NHÂN	Chỉ tồn tại ở bảng gốc; mọi view phân tích dùng cột ma_khach_gia_danh (HMAC-SHA256, tính trong pipeline). Vai trò nha_phan_tich không có quyền SELECT trên bảng này.
fact_ticket.tre_han	SMALLINT	Cờ 0/1: phiếu được đóng sau hạn xử lý hay không. Lưu 0/1 để cộng dồn được; tỉ lệ trễ hạn tính bằng AVG lúc truy vấn.	Chỉ 0 hoặc 1.	Không (NOT NULL)	NỘI BỘ	Không cần xử lý. Vai trò nha_phan_tich có quyền đọc.

B5. Checklist tự kiểm [DA]

- ☐ Đã phân loại mọi cột trong kho dữ liệu thành bốn mức và ghi vào Data dictionary.
- ☐ Mọi cột định danh cá nhân trong kho phân tích đã được giả danh hoá hoặc loại bỏ.
- ☐ Giả danh hoá dùng HMAC với khoá bí mật lấy từ biến môi trường, tính trong pipeline; không băm "trần" và không lưu khoá trong CSDL.

- ☐ Có ngưỡng nhóm tối thiểu cho báo cáo tổng hợp, và ngưỡng đó được ghi trong Data dictionary kèm lý do.
- ☐ Có ít nhất hai vai trò truy cập với quyền khác nhau, và dashboard dùng vai trò hạn chế.
- ☐ Dashboard không kết nối bằng tài khoản chủ sở hữu cơ sở dữ liệu.
- ☐ Thông báo lỗi của dashboard không in chuỗi kết nối, tên bảng hay vết ngăn xếp.
- ☐ Không commit tệp dữ liệu chứa thông tin cá nhân vào repo; dữ liệu mẫu trong repo đã được giả danh hoá.
- ☐ Pipeline doc có đủ sáu mục, trong đó mục hạn chế đã biết nêu ảnh hưởng nghiệp vụ cụ thể.
- ☐ Data dictionary có mọi cột của mọi bảng đích, mỗi cột đủ sáu trường như mẫu ở mục B4.
- ☐ Có ảnh chụp minh chứng: kết quả đăng nhập bằng vai trò nha_phan_tich rồi SELECT trên dim_customer bị từ chối (permission denied).

PHẦN C — Track AI: Model card, báo cáo thiên lệch và AI có trách nhiệm

C1. Model card — mười mục, là thành phần 6 của Bài tập 3

Model card là tài liệu đi kèm một mô hình, viết cho người sẽ DÙNG mô hình đó chứ không phải cho người xây nó. Mục tiêu: người đọc quyết định được mô hình này có phù hợp với việc của họ hay không, mà không cần đọc mã nguồn.

Mục	Nội dung tối thiểu	Ví dụ với mô hình phân loại sự cố Mekong Mobile
1. Thông tin cơ bản	Tên mô hình, phiên bản, ngày tạo, người tạo, cách liên hệ	phan-loai-su-co v1.0 · 15/11/2026 · sinh viên <tên>, MSSV <số> · liên hệ qua giảng viên hướng dẫn
2. Mục đích sử dụng dự kiến	Bài toán nó giải, ai dùng, dùng trong quy trình nào	GỢI Ý nhóm lỗi cho nhân viên tiếp nhận khi họ nhập mô tả sự cố, để giảm thời gian nhập liệu. Nhân viên là người quyết định cuối cùng.
3. Trường hợp KHÔNG nên dùng	Danh sách cụ thể các việc mô hình này không phù hợp	KHÔNG dùng để tự động phân công kỹ thuật viên · KHÔNG dùng để quyết định phiếu có được bảo hành hay không · KHÔNG dùng cho ngôn ngữ khác tiếng Việt · KHÔNG dùng để đánh giá năng lực nhân viên
4. Dữ liệu huấn luyện	Nguồn, số mẫu, khoảng thời gian, cách chia, phân bố nhãn	4.000 mô tả sự cố từ hệ thống nội bộ, giai đoạn 2024–2026 · chia 70/15/15 phân tầng, hạt giống 42 · phân bố nhãn: KHÁC 31%, MÀN_HÌNH 22%, PIN 16%, SẠC 13%, PHẦN_MỀM 9%, NƯỚC_VÀO 9%
5. Kiến trúc và cách xử lý	Mô hình gì, đặc trưng gì, các bước tiền xử lý	TF-IDF 1–2 gram (min_df=2) + hồi quy logistic có cân bằng trọng số lớp · tiền xử lý: chữ thường, GIỮ dấu tiếng Việt, thay số điện thoại và serial bằng thẻ
6. Chỉ số đánh giá	Chỉ số nào, đo trên tập nào, kèm điều kiện đo và độ dao động	F1 macro trên tập test = 0,750 (dữ liệu v1.0, hạt giống 42) · kiểm định chéo 5 phần trên tập train = 0,762 ± 0,031 · F1 theo nhãn: MÀN_HÌNH 0,83 · PIN 0,79 · SẠC 0,74 · PHẦN_MỀM 0,71 · NƯỚC_VÀO 0,69 (precision 0,86 · recall 0,57) · KHÁC 0,74
7. Mô hình cơ sở để so sánh	Con số của mô hình đơn giản nhất	Đoán nhãn đa số: F1 macro = 0,079. Mô hình hiện tại cải thiện đáng kể so với mức sàn này.
8. Thiên lệch đã đo	Cách nhóm, chênh lệch giữa nhóm tốt nhất và kém nhất, giả thuyết nguyên nhân	Theo trung tâm: chênh 0,06 (chấp nhận được) · theo độ dài mô tả: mô tả dưới 10 từ có F1 = 0,61 so với 0,81 ở mô tả trên 25 từ, chênh 0,20 — vượt ngưỡng, xem mục 9
9. Rủi ro và giới hạn	Ít nhất hai rủi ro cụ thể kèm biện pháp giảm thiểu	(a) Mô tả quá ngắn cho kết quả kém: giảm thiểu bằng cách nâng ngưỡng tin cậy lên 0,60 với mô tả dưới 10 từ và chuyển sang nhân viên tự chọn. (b) Nhãn NƯỚC_VÀO có recall 57%, và bỏ sót ca này gây hỏng nặng thiết bị: giảm thiểu bằng cách luôn hiển thị NƯỚC_VÀO trong top 3 khi mô tả chứa các từ liên quan tới nước.
10. Bảo trì	Khi nào cần huấn luyện lại, dấu hiệu mô hình xuống cấp, ai chịu trách nhiệm	Huấn luyện lại khi tỉ lệ nhân viên sửa gợi ý vượt 35% trong một tháng, hoặc khi có nhóm lỗi mới được thêm vào danh mục · theo dõi bằng cách ghi nhật ký gợi ý và lựa chọn cuối của nhân viên

Ba mục quyết định điểm của Model card

Mục 3 (trường hợp KHÔNG nên dùng) là mục phân biệt Model card thật với một bản mô tả kỹ thuật. Nó chứng minh sinh viên hiểu giới hạn của thứ mình làm ra. Một Model card không có mục này gần như không được điểm.

Mục 8 và 9 (thiên lệch và rủi ro) phải có SỐ và có BIỆN PHÁP CỤ THỂ. "Mô hình có thể có thiên lệch" không được điểm; "chênh 0,20 giữa mô tả ngắn và dài, giảm thiểu bằng cách nâng ngưỡng tin cậy cho mô tả ngắn" thì được điểm.

Mục 6 phải có điều kiện đo. Một con số không có phiên bản dữ liệu, hạt giống và tập đo là con số không kiểm chứng được — vi phạm nguyên tắc thứ ba của học phần.

C2. Checklist AI có trách nhiệm — bảy câu hỏi tự trả lời

Bảy câu dưới đây là phiên bản rút gọn, phù hợp phạm vi học phần, của các khung AI có trách nhiệm hiện hành. Trả lời cả bảy câu bằng văn bản, mỗi câu hai đến ba dòng, và đưa vào Bài tập 3 như một phần của Model card.

- ☐ 1. Người bị ảnh hưởng bởi một dự đoán SAI của mô hình này là ai, và họ mất gì? Trả lời cụ thể, không trả lời "người dùng".
- ☐ 2. Người đó có biết là có một hệ thống AI tham gia vào quyết định không? Nếu không, vì sao không, và có nên cho họ biết không?
- ☐ 3. Người đó có cách nào phản đối hoặc yêu cầu xem lại một kết quả không? Nếu không có, đó là một hạn chế phải ghi rõ.
- ☐ 4. Mô hình có hoạt động kém hơn với một nhóm người hoặc một nhóm dữ liệu nào không? Sinh viên đã ĐO chưa, hay chỉ đang đoán?
- ☐ 5. Mô hình có dùng đặc trưng nào mà việc dùng nó là không phù hợp về mặt đạo đức (giới tính, tuổi, nơi ở, khi những thông tin đó không liên quan tới bài toán)?
- ☐ 6. Dữ liệu huấn luyện có được thu thập một cách hợp lệ, và người cung cấp dữ liệu có biết dữ liệu của họ được dùng cho việc này không?
- ☐ 7. Nếu mô hình này ngừng hoạt động, hệ thống còn chạy được không, và ai nhận biết được là nó đã ngừng?

Câu 4 và câu 5 là hai câu bị trả lời hình thức nhiều nhất

Câu 4 — "mô hình có thể có thiên lệch" là chưa trả lời. Câu trả lời đúng nêu cách nhóm cụ thể, số liệu chênh lệch, và kết luận. Nếu sinh viên chưa đo thì hãy trả lời trung thực "tôi chưa đo theo chiều này" — trung thực được điểm, đoán bừa thì không.

Câu 5 — nhiều mô hình dùng đặc trưng gián tiếp mà không nhận ra. Ví dụ: dùng tỉnh thành để dự đoán khả năng bảo hành có thể trở thành phân biệt theo địa phương. Hãy đi qua từng đặc trưng và hỏi "nếu người dùng biết mình bị đánh giá dựa trên đặc trưng này, họ có thấy hợp lý không?".

Liên hệ đề thi: câu 4b của đề thi cuối kỳ hỏi về một trách nhiệm pháp lý hoặc đạo đức khi phát hành mô hình ra công chúng. Bảy câu này chính là nguồn để sinh viên trả lời.

C3. Ba nghĩa vụ minh bạch cần biết — cập nhật tới tháng 10/2026

Nội dung dưới đây ở mức NHẬN BIẾT, để sinh viên biết chỗ cần tra cứu khi làm sản phẩm thật. Đây không phải tư vấn pháp lý và không thay thế việc tra cứu văn bản hiện hành. Quy định trong lĩnh vực này thay đổi nhanh — hãy kiểm tra lại tại nguồn chính thức trước khi dựa vào nó.

Nghĩa vụ	Nội dung ở mức nhận biết	Áp dụng vào prototype của sinh viên thế nào
Cho người dùng biết họ đang tương tác với AI	Theo Điều 50 của EU AI Act (Quy định (EU) 2024/1689), hệ thống tương tác trực tiếp với người (như chatbot) phải cho người đó biết đang tương tác với AI. Nghĩa vụ này áp dụng từ 02/8/2026; gói sửa đổi Digital Omnibus về AI không lùi mốc này. Luật Trí tuệ nhân tạo của Việt Nam (hiệu lực từ 01/3/2026) cũng yêu cầu bảo đảm người dùng nhận biết được họ đang tương tác với hệ thống AI.	Nếu sản phẩm của sinh viên có chatbot hoặc phần trả lời tự động, thêm một dòng thông báo rõ ràng. Nếu chỉ là mô hình gợi ý nội bộ cho nhân viên, ghi rõ trong giao diện rằng đây là gợi ý tự động.
Đánh dấu nội dung do AI sinh ra	Nội dung do AI sinh (văn bản, ảnh, âm thanh, video) phải được đánh dấu ở dạng máy đọc được; deepfake phải được công bố rõ. Theo Digital Omnibus, hệ thống sinh nội dung đã đưa ra thị trường TRƯỚC 02/8/2026 có thời gian chuyển tiếp tới 02/12/2026; hệ thống mới thì áp dụng ngay.	Nếu prototype của sinh viên sinh nội dung, thêm dấu hiệu nhận biết. Nếu không sinh nội dung thì ghi "không áp dụng" kèm lý do.
Nêu rõ giới hạn sử dụng	Người phát hành một mô hình có trách nhiệm nêu rõ mô hình dùng cho việc gì và KHÔNG dùng cho việc gì, để người khác không dùng sai mục đích.	Đây chính là mục 3 của Model card. Đây là nghĩa vụ áp dụng cho MỌI prototype AI, kể cả prototype nội bộ.

Lưu ý quan trọng về phạm vi pháp lý

EU AI Act là quy định của Liên minh châu Âu. Sản phẩm của sinh viên có thể chịu quy định này nếu phục vụ người dùng ở EU.

Ở Việt Nam: Luật Bảo vệ dữ liệu cá nhân (số 91/2025/QH15, hiệu lực từ 01/01/2026) và Luật Trí tuệ nhân tạo (thông qua 10/12/2025, hiệu lực từ 01/3/2026, phân loại hệ thống AI theo ba mức rủi ro cao · trung bình · thấp). Khi làm sản phẩm thật, hãy tra cứu văn bản và nghị định hướng dẫn hiện hành.

Điều học phần này yêu cầu ở sinh viên không phải là thuộc luật, mà là: nhận ra được khi nào một tình huống có vấn đề, biết rằng có quy định về nó, và biết chỗ tra cứu.

Nguồn chính thức để tra cứu: digital-strategy.ec.europa.eu (EU AI Act) · vanban.chinhphu.vn (văn bản pháp luật Việt Nam). Mốc thời gian trong tài liệu này đúng tại tháng 10/2026 và cần được kiểm tra lại mỗi học kỳ.

C4. Bảo mật cho dịch vụ mô hình — bốn mục riêng của track AI

Mục	Rủi ro	Cách xử lý ở Bài tập 3
Giới hạn kích thước đầu vào	Một mô tả dài vài megabyte làm dịch vụ mô hình treo hoặc hết bộ nhớ	Đặt độ dài tối đa trong lược đồ kiểm tra dữ liệu vào (ví dụ 2.000 ký tự) và trả 422 khi vượt

Mục	Rủi ro	Cách xử lý ở Bài tập 3
Giới hạn số lần gọi	Ai đó gọi liên tục làm dịch vụ mô hình chiếm hết tài nguyên và nghiệp vụ chính bị ảnh hưởng	Ở mức prototype: đặt thời gian chờ tối đa khi gọi và bảo đảm suy giảm có kiểm soát. Giới hạn số lần gọi đầy đủ thuộc CĐTN2
Không ghi dữ liệu người dùng vào nhật ký	Mô tả sự cố có thể chứa số điện thoại, tên khách, số serial — ghi hết vào nhật ký là làm rò rỉ dữ liệu cá nhân	Nhật ký chỉ ghi độ dài mô tả, nhãn dự đoán, độ tin cậy và thời gian xử lý. KHÔNG ghi nội dung mô tả
Không để lộ thông tin mô hình qua thông báo lỗi	Vết ngăn xếp để lộ đường dẫn tệp, phiên bản thư viện, cấu trúc mô hình	Bắt mọi ngoại lệ ở tầng ngoài, trả về thông báo chung; chi tiết chỉ ghi vào nhật ký phía máy chủ

C5. Checklist tự kiểm [AI]

- ☐ Model card có đủ mười mục ở bảng C1.
- ☐ Mục 3 (trường hợp KHÔNG nên dùng) có ít nhất ba mục cụ thể.
- ☐ Mục 6 (chỉ số) ghi kèm điều kiện đo: phiên bản dữ liệu, hạt giống, tập đo, độ dao động.
- ☐ Mục 8 (thiên lệch) có số liệu đo thật theo ít nhất hai cách nhóm.
- ☐ Mục 9 (rủi ro) có ít nhất hai rủi ro, mỗi rủi ro kèm một biện pháp giảm thiểu cụ thể làm được.
- ☐ Đã trả lời cả bảy câu của checklist AI có trách nhiệm, mỗi câu hai đến ba dòng.
- ☐ Đã trả lời trung thực câu 4 — nếu chưa đo theo một chiều nào thì ghi rõ là chưa đo.
- ☐ Dịch vụ mô hình giới hạn độ dài đầu vào và trả 422 khi vượt.
- ☐ Nhật ký dịch vụ mô hình KHÔNG ghi nội dung mô tả của người dùng.
- ☐ Thông báo lỗi của dịch vụ mô hình không để lộ đường dẫn tệp hay phiên bản thư viện.
- ☐ Nếu prototype có phần tương tác trực tiếp với người, đã có thông báo cho biết đó là AI.

So sánh nhanh ba track — bảng đối chiếu để tự kiểm

Mục checklist bảo mật	Track SE	Track DA	Track AI
1. Không có secret trong repo	JWT_SECRET, chuỗi kết nối, khoá API đều trong .env	Chuỗi kết nối kho dữ liệu, khoá giả danh hoá (PSEUDO_KEY) đều trong .env	Khoá API của dịch vụ bên ngoài (nếu có) trong .env
2. Kiểm tra dữ liệu vào	Validator ở tầng ứng dụng; ba lệnh grep chống injection	Kiểm chất lượng dữ liệu trước khi nạp; không nạp nếu dưới ngưỡng	Lược đồ kiểm tra đầu vào của dịch vụ dự đoán; giới hạn độ dài
3. Kiểm soát truy cập	Kiểm chủ sở hữu ở tầng service; test case theo A01 OWASP	Vai trò cơ sở dữ liệu; view đã giả danh hoá; dashboard dùng vai trò hạn chế	Dịch vụ mô hình chỉ gọi được từ mạng nội bộ của compose
4. Băm hoặc mã hoá	Mật khẩu băm bằng bcrypt hoặc argon2	Cột định danh cá nhân giả danh hoá bằng HMAC có khoá bí mật	Không áp dụng trực tiếp; ghi rõ lý do
5. Thông báo lỗi không lộ nội bộ	Bản đồ mã nghiệp vụ sang mã HTTP; không trả vết ngăn xếp	Dashboard không in chuỗi kết nối; Streamlit đặt client.showErrorDetails = false	Không lộ đường dẫn tệp mô hình và phiên bản thư viện
6. Nhật ký an toàn	Hàm che các trường mật khẩu, token trước khi ghi	Nhật ký pipeline không ghi nội dung dòng dữ liệu cá nhân	Nhật ký chỉ ghi độ dài, nhãn, độ tin cậy — không ghi nội dung
7. Ghim phiên bản phụ thuộc	package-lock.json đã commit; npm audit ở mức cơ bản	requirements.txt dùng dấu ==; pip-audit ở mức cơ bản	requirements.txt dùng dấu ==; phiên bản thư viện phải khớp với lúc lưu mô hình
Tài liệu kỹ thuật (thành phần 6)	API docs: mỗi endpoint đủ bảy mục, có ví dụ THẬT	Pipeline doc sáu mục + Data dictionary mỗi cột sáu trường	Model card mười mục + bảy câu AI có trách nhiệm

Ba việc làm ngay trong tuần này, trước workshop buổi 16 (nộp Bài tập 3)

1. Rà secret trước tiên — cả trong lịch sử Git, vì repo PUBLIC. Chạy git log --all --oneline -- .env (phải KHÔNG ra dòng nào) và git log -p --all | grep -inE "password|secret|api_key|token" | head (chạy được trên repo nhỏ). Nếu tìm thấy, đổi mọi secret liên quan ngay và ghi việc đó vào tài liệu. Đây là mục số 1 của checklist và là mục nghiêm trọng nhất.

2. Điền checklist bảy mục bằng cách CHẠY THỬ, không bằng cách nhớ lại. Mỗi mục phải có một minh chứng cụ thể: một lệnh đã chạy kèm **ảnh chụp kết quả**, hoặc một đường dẫn tệp. Ghi "chưa làm" trung thực không bị trừ điểm.

3. Viết tài liệu kỹ thuật của track SONG SONG với việc kiểm bảo mật, không để tới cuối. Nhiều mục của tài liệu (mã lỗi có thể trả về, hạn chế đã biết, giới hạn sử dụng) chính là kết quả của việc kiểm bảo mật — làm cùng lúc thì nhanh hơn nhiều.

VAN LANG UNIVERSITY · FACULTY OF INFORMATION TECHNOLOGY
SPECIALIZED GRADUATION TOPIC 1 · SEMESTER 1, ACADEMIC YEAR 2026 – 2027

Self-study guide, Session 15 — Security, access control and technical documentation by specialization

HTTPS, CORS, JWT, input validation and API docs [SE] · Access control and anonymisation of sensitive data [DA] · Model card, bias report and a responsible-AI checklist [AI]. Each part has a filled-in documentation template and a self-assessment security checklist.

The output of this guide is components 5 and 6 of Assignment 3 — the self-assessed security checklist and the technical documentation for your track. Finish them in the Session 16 workshop; submit at the end of Session 16.

Prepared by: **Dr. Nguyễn Trí Hải** · Department of Software Engineering

For internal circulation · Self-study series for the specialization part (30% of Session 15 theory time) · Legal content updated to 10/2026 and must be re-checked every semester. Bilingual edition: this is the English version. In the sample code, identifiers, user-facing strings and log messages stay in Vietnamese because the case-study system serves Vietnamese users; code comments are translated.

How to use this guide

Taught by the lecturer in class (Session 15)	Read by students in this guide
The three levels of reproducibility and why Assignment 3 requires level 3; a complete docker-compose with healthchecks and sample data; the clean-machine test; environment variables and the four places a secret must never be; the five sections of the Deployment Guide (in English); OWASP Top 10:2025 at awareness level; the seven mandatory security checklist items.	How to implement each security item on your own track's stack, and the documentation template for your track: API docs [SE] · Pipeline doc and Data dictionary [DA] · Model card [AI].

The seven mandatory security checklist items — recap from the lecture

1. No secret anywhere in the repo (passwords, API keys, connection strings, tokens).
2. Input is validated in the application layer, not only by database constraints.
3. Access control: users can read and change only the data they own.
4. Passwords and sensitive data are hashed or encrypted, never stored in readable form.
5. Error messages do not reveal internals (stack traces, table names, connection strings).
6. System logs do not record passwords, tokens or unnecessary personal data.
7. External dependencies are pinned to exact versions and checked for vulnerability warnings at a basic level.

How to fill it in: mark each item DONE / NOT DONE / NOT APPLICABLE, with one line of evidence (a file path, a command you ran with its result, **a screenshot of the run**, or the reason if it is "not applicable"). An honest "not done" costs NO marks; "done" without evidence loses marks.

The Assignment 3 repo must be PUBLIC (the lecturer does not accept collaborator invitations). Everything in the Git history is therefore public — item 1 of the checklist is the most serious one.

Scope boundary — read this so you do not do extra work

Content	Which course it belongs to
The seven checklist items above, self-assessed with evidence	Specialized Graduation Topic 1 — mandatory in Assignment 3
Recognising the ten OWASP Top 10:2025 risk categories and self-checking the five priority ones	Specialized Graduation Topic 1 — awareness level
Full security review, penetration testing, threat modelling	Specialized Graduation Topic 2
Managing secrets with a dedicated system (vault), automatic key rotation	Specialized Graduation Topic 2
Running full monitoring, security incident response, compliance with international standards (designing four-part alerts belongs to this course — session 15)	Specialized Graduation Topic 2 and advanced courses

PART A — Track SE: HTTPS, CORS, JWT, input validation and API docs

A1. HTTPS within the scope of this course

Your prototype runs on localhost, so it does not use real HTTPS. What you must show in Assignment 3 is that you KNOW where HTTPS is needed and have prepared for it — not that you have installed a certificate.

Task	What to do for Assignment 3	Not required in this course
Redirect to HTTPS	State in the Deployment Guide: "in a real deployment, place the app behind a proxy with a TLS certificate; the app reads the X-Forwarded-Proto header to know the original request was HTTPS".	Installing and configuring a real TLS certificate
Secure cookies	If you use cookies: set HttpOnly and SameSite now; set Secure conditionally from an environment variable so localhost still works.	Configuring HSTS, certificate pinning
Never send secrets in the URL	Check now: tokens, passwords and keys never appear in the query string, because query strings end up in server logs and browser history.	None

Conditional secure-cookie configuration — works on localhost too

```
const LA_MOI_TRUONG_THAT = process.env.NODE_ENV === 'production';

res.cookie('refresh_token', token, {
  httpOnly: true,                    // JavaScript code CANNOT read it
  secure:   LA_MOI_TRUONG_THAT,      // sent only over HTTPS in a real environment
  sameSite: LA_MOI_TRUONG_THAT ? 'strict' : 'lax',
  maxAge:   7 * 24 * 3600 * 1000,
  path:     '/api/auth',             // limit the path the cookie is sent with
});

// In the Deployment Guide, state clearly:
//   "Set NODE_ENV=production in a real deployment. The cookie is then sent
//    only over HTTPS. If deployed behind a proxy, enable app.set('trust proxy', 1)
//    so that Express reads X-Forwarded-Proto correctly."
```

A2. CORS — configure it correctly, do not open it to every origin

Correct CORS configuration for Assignment 3

```
const cors = require('cors');

// WRONG - open to every origin; must NOT be used with cookie-based auth
// app.use(cors());
// app.use(cors({ origin: '*', credentials: true }));   // browsers reject this anyway

// RIGHT - an allow-list of origins, read from an environment variable
const NGUON_CHO_PHEP = (process.env.CORS_ORIGINS || 'http://localhost:5173')
  .split(',').map(s => s.trim()).filter(Boolean);

app.use(cors({
  origin: (nguon, cb) => {
    // an empty origin means the request is not from a browser (curl, mobile app)
    if (!nguon || NGUON_CHO_PHEP.includes(nguon)) return cb(null, true);
    cb(new Error('Nguồn không được phép bởi cấu hình CORS'));
  },
  credentials: true,
  methods: ['GET', 'POST', 'PATCH', 'DELETE'],
  allowedHeaders: ['Content-Type', 'Authorization', 'X-Request-Id'],
  maxAge: 600,
}));

// .env.example:
//   CORS_ORIGINS=http://localhost:5173,http://localhost:3000
```

Three things to understand about CORS so you can answer questions in Session 16

CORS is a BROWSER mechanism, not a server security mechanism. It stops website A from calling your API with the user's credentials. It does NOT stop anyone calling your API with curl. So CORS does not replace authentication and access control.

origin: "*" together with credentials: true is an invalid configuration. Browsers reject it. If you open to every origin you must drop credentials, and then cookies are not sent.

Read the allow-list from an environment variable. Hard-coding the frontend address in source code is a configuration bug: when you deploy to another domain you have to change the code and rebuild.

A3. JWT — four common mistakes and how to avoid them

Mistake	Why it is dangerous	The right way
Secret key written in source code, or too short	Whoever has the key can mint a valid token for any user, including administrators	Generate a random key of at least 32 bytes, keep it in an environment variable; .env.example holds only a sample value
Expiry not checked, or far too long	A leaked token stays usable forever; it cannot be revoked	Short-lived access tokens (15–60 minutes); if longer sessions are needed, add a refresh token stored in an HttpOnly cookie
Trusting token contents without verifying the signature, or accepting the "none" algorithm	Anyone can change the role in the token to admin	Always use the verify function with an explicit list of algorithms; never use decode to make authorisation decisions
Putting personal data in the token payload	A JWT payload is only base64-encoded; anyone can read it	The payload holds only the user id, role and expiry. No email, phone number or address in the token

src/middleware/auth.js — verifying a token correctly

```
const jwt = require('jsonwebtoken');

const KHOA = process.env.JWT_SECRET;
if (!KHOA || KHOA.length < 32) {
  // Stop at start-up; never let the system run with an unsafe configuration
  throw new Error('JWT_SECRET phải có ít nhất 32 ký tự');
}

function xacThuc(req, res, next) {
  const header = req.headers.authorization || '';
  const token = header.startsWith('Bearer ') ? header.slice(7) : null;
  if (!token) return res.status(401).json({ error: 'CHUA_DANG_NHAP' });
  try {
    // SPECIFY the algorithm - never let the library pick it from the token header
    const payload = jwt.verify(token, KHOA, { algorithms: ['HS256'] });
    req.user = { id: payload.sub, vai_tro: payload.vai_tro };
    next();
  } catch (e) {
    // do NOT return the library's error details to the caller
    return res.status(401).json({ error: 'TOKEN_KHONG_HOP_LE' });
  }
}

// Access control by OWNER - checklist item 3,
// and the most common vulnerability in student prototypes (OWASP A01)
function chiChuSoHuu(layChuSoHuu) {
  return async (req, res, next) => {
    const chuSoHuuId = await layChuSoHuu(req);
    if (chuSoHuuId === null || chuSoHuuId === undefined)
      return res.status(404).json({ error: 'KHONG_TIM_THAY' });
    // Compare like with like: the token id may be a string, the database id a number
    if (String(chuSoHuuId) !== String(req.user.id) && req.user.vai_tro !== 'QUAN_TRI')
      return res.status(403).json({ error: 'KHONG_CO_QUYEN' });
    next();
  };
}

module.exports = { xacThuc, chiChuSoHuu };
```

A4. Input validation — preventing injection in practice

Type	How the system is attacked	Prevention for Assignment 3	How to check it yourself
SQL injection	The query string is concatenated with user data	Always use parameterised queries ($1, ?) or an ORM. NEVER concatenate SQL strings.	grep for string concatenation in queries; type ' OR 1=1 -- into a search box
Cross-site scripting	User data is inserted into HTML and executed as code	Let the framework escape output (React and Vue do this by default); never use innerHTML or dangerouslySetInnerHTML with user data	grep for innerHTML and dangerouslySetInnerHTML; type a script tag into a text field and see whether it is displayed or executed
Command injection	User data is passed into a system command	Do not call system commands with user data. If you must, pass an argument array, not a shell string.	grep for exec, spawn, os.system, subprocess with shell=True
Path traversal	The user sends ../../etc/passwd as a file name	Do not accept paths from users. For downloads, use an id, not a file name.	Call the download endpoint with a file name containing ../

Three grep commands, three minutes, most problems found

1. grep -rnE '(SELECT|INSERT|UPDATE|DELETE).*(\+ *[a-zA-Z_]|\$\{)' src/ — finds SQL built by concatenation or by inserting variables into template strings.

2. grep -rnE "innerHTML|dangerouslySetInnerHTML|v-html" src/ — finds places that insert raw HTML.

3. grep -rnE 'exec\(|execSync|spawn\(|os\.system|shell=True' src/ — finds calls to system commands.

Screenshot the output of the three commands and record it under item 2 of the security checklist as evidence. Empty output is good evidence; if there is output, review each hit and fix it or explain why it is safe.

A5. API docs — component 6 of Assignment 3 for track SE

API docs must describe what the system ACTUALLY does. Documenting an endpoint that does not exist is a consistency error and loses marks under criterion 3 of the rubric.

Approach	Advantages	Requirement to pass Assignment 3
OpenAPI / Swagger	Produces an interactive documentation page; some frameworks generate it from the code	An openapi.yaml or openapi.json file in the repo; opens in Swagger UI or pastes into editor.swagger.io without errors
Postman collection	Easiest; doubles as a manual API test suite	Export the .postman_collection.json file and commit it; every request has a saved example response from a real run
Hand-written Markdown file	No tools needed; readable directly on GitHub	Every endpoint has the seven sections in the table below; at least one REAL request and response example per endpoint

Seven mandatory sections for EVERY endpoint	Example
1. Method and path	POST /api/tickets
2. One-sentence business description	Receives a device for warranty service, creates a ticket and computes the due date from the priority level.
3. Authentication and permission	Token required; role NHAN_VIEN_TIEP_NHAN or QUAN_TRI.
4. Parameters and request body	A table of every field: name, type, required or not, constraints. Plus one real JSON example.
5. Success response body	Status 201 with a REAL JSON example copied from an actual run, not an invented one.
6. Table of possible error codes	400 missing required field · 401 not logged in · 403 no permission · 404 customer not found · 409 device already has an open ticket · 422 technician overloaded.
7. Notes on limits or special behaviour	Calling again with the same serial while the previous ticket is still open returns 409 instead of creating a second ticket.

A6. Self-check list [SE]

- ☐ I ran the three grep commands in A4 and recorded the results in the security checklist.
- ☐ JWT_SECRET comes from an environment variable, is at least 32 characters, and the system stops if it is missing.
- ☐ The token verification function specifies the algorithm and never uses decode for authorisation.
- ☐ The token payload holds only id, role and expiry — no personal data.
- ☐ Access control by owner is in place, and I tested it with a test case as in OWASP A01.
- ☐ CORS is open only to origins listed in an environment variable; no origin: "*" together with credentials.
- ☐ Cookies (if used) have HttpOnly and SameSite; Secure is set from an environment variable.
- ☐ No secret, token or password appears in any URL.
- ☐ Error messages contain no stack trace, table name or connection string.
- ☐ The API docs cover every existing endpoint, each with all seven sections and REAL examples.
- ☐ The Deployment Guide has a section explaining how to enable HTTPS in a real deployment.

PART B — Track DA: Access control and anonymisation of sensitive data

B1. Classify the data — do this before anything else

You cannot protect data until you know which data needs protecting. The first step is to go through every column in your data warehouse and assign one of four levels. It takes about thirty minutes and underpins every later decision.

Level	Definition	Examples in the Mekong Mobile dataset	Handling
PUBLIC	Harmless if published	product_name, thuong_hieu, nhom_san_pham, store_name, thanh_pho	No handling needed
INTERNAL	For use inside the organisation; does not identify a person	so_gio_xu_ly, tre_han, chi_phi_linh_kien, category_name, aggregated figures	Role-based access control
PERSONALLY IDENTIFYING	Identifies a specific person	ho_ten, so_dien_thoai, email, dia_chi, serial_no (tied to one person's device)	Anonymise or pseudonymise in every analytical table; keep the original only where the business truly needs it
SENSITIVE	Leakage causes serious harm	None in this dataset; if your project has national ID numbers, health data or personal financial data, they belong here	Encrypt at rest; restrict access to the minimum; log every access

The principle of MINIMISATION applied to the warehouse

An analytical warehouse usually does NOT need full names, phone numbers and addresses. It needs to know "this is the same customer" and "which province this customer is in".

So: replace ho_ten and so_dien_thoai with a stable pseudonymous code; keep tinh_thanh but drop the house number and street; keep an age band instead of a date of birth.

The test: for every personally identifying column in the warehouse, ask "which report needs exactly this column in its original form?". If no report does, that column should not be in the warehouse in its original form.

Record the result in the Data dictionary: every column gets two extra fields, the classification level and the handling. This is the most valuable part of your Data dictionary and what sets top-scoring work apart.

B2. Three anonymisation techniques — the right technique for the right purpose

Technique	How it works	Can records of the same person still be linked?	When to use it
Pseudonymisation	Replace the identifying value with a stable code generated by HMAC-SHA256 with a secret key read from an environment variable	YES — the same person always gets the same code	Analytical warehouse: you need to link one customer's history without knowing who it is
Masking	Keep part of the value and hide the rest: 0912***678	Not reliably — several people can share the same masked form	Customer-support screens: staff need a quick check with the customer on the phone
Aggregation	Store no individual rows, only figures per group; and hide groups with too few records	NO	Reports shared externally, or dashboards for people not allowed to see personal data

src/transform/gia_danh.py — pseudonymise IN THE PIPELINE; the key never enters the database

```
import hashlib, hmac, os

# The secret key comes from an environment variable - NOT in the code, NOT in the database.
# Never use a "bare" hash: a phone number has only 10 digits, so an unkeyed hash
# is easily reversed by trying every number.
KHOA = os.environ["PSEUDO_KEY"].encode()      # .env.example: PSEUDO_KEY=doi-gia-tri-nay
if len(KHOA) < 32:
    raise SystemExit("PSEUDO_KEY phải có ít nhất 32 ký tự")

def gia_danh(gia_tri: str | None) -> str | None:
    """The same phone number always yields the same code - customer history stays
linkable."""
    if not gia_tri:
        return None
    return hmac.new(KHOA, gia_tri.encode(), hashlib.sha256).hexdigest()

# In the transform step:
#   df["ma_khach_gia_danh"] = df["sdt_chuan"].map(gia_danh)
```

sql/030_view_phan_tich.sql — analytical view and an aggregate view with a group threshold

```
-- VIEW for analysts: NO name, phone number or full address
CREATE OR REPLACE VIEW v_khach_hang_phan_tich AS
SELECT
  dc.customer_sk,                     -- surrogate key, no business meaning
  dc.ma_khach_gia_danh,               -- computed in the pipeline (gia_danh.py)
  dc.tinh_thanh,                      -- keep the province, DROP house number and street
  date_trunc('month', dc.ngay_tao)::DATE AS thang_tao,   -- coarser time granularity
  dc.nhom_khach                       -- business category, not identifying
FROM dim_customer dc;

-- Aggregate VIEW for widely shared dashboards: HIDE groups with too few customers
CREATE OR REPLACE VIEW v_bao_cao_tinh AS
SELECT k.tinh_thanh,
       COUNT(DISTINCT k.customer_sk)                              AS so_khach,
       ROUND(COUNT(f.ticket_sk)::NUMERIC
             / COUNT(DISTINCT k.customer_sk), 2)                  AS so_phieu_tb
FROM      v_khach_hang_phan_tich k
LEFT JOIN fact_ticket f ON f.customer_sk = k.customer_sk
GROUP BY  k.tinh_thanh
HAVING    COUNT(DISTINCT k.customer_sk) >= 20;  -- MINIMUM GROUP SIZE: provinces with
                                                -- fewer than 20 customers are hidden,
                                                -- since tiny groups reveal individuals
```

Why the "minimum group size" threshold is the most important point of B2

An aggregate report that looks safe can still expose an individual: if a province has only one customer, "the average revenue of province X" is that person's revenue.

Combining several aggregate reports can also reveal an individual: knowing the total of a group of 5 and the total of 4 of them gives you the fifth person.

What to do for Assignment 3: set a minimum group size (20 is reasonable for this data), hide every group below it, and STATE the threshold and its rationale in the Data dictionary.

The full techniques (differential privacy, l-diversity) belong to advanced courses. Here you only need to recognise the risk and apply the simple measure correctly.

B3. Role-based access control in the warehouse

sql/040_quyen_truy_cap.sql — three roles, three levels of access

```
-- Role 1: analyst - reads only the pseudonymised VIEWs
CREATE ROLE nha_phan_tich;
GRANT USAGE  ON SCHEMA public TO nha_phan_tich;
GRANT SELECT ON v_khach_hang_phan_tich, v_bao_cao_tinh, fact_ticket TO nha_phan_tich;
-- NO grant on dim_customer (the table with original data). The view can still read
-- dim_customer because a view runs with the privileges of its owner.

-- Role 2: business staff - see only their own centre's tickets
CREATE ROLE nhan_vien_nghiep_vu;
GRANT USAGE  ON SCHEMA public TO nhan_vien_nghiep_vu;
GRANT SELECT ON dim_customer, fact_ticket TO nhan_vien_nghiep_vu;

-- Restrict by ROW with Row Level Security.
-- Note: once RLS is on, any role WITHOUT a policy sees 0 rows,
-- so declare a policy for BOTH roles that read fact_ticket.
ALTER TABLE fact_ticket ENABLE ROW LEVEL SECURITY;
CREATE POLICY chi_trung_tam_cua_minh ON fact_ticket FOR SELECT
  TO nhan_vien_nghiep_vu
  USING (center_sk = current_setting('app.center_sk')::INT);
CREATE POLICY phan_tich_doc_tat_ca ON fact_ticket FOR SELECT
  TO nha_phan_tich
  USING (true);

-- Role 3: the pipeline account - minimum privileges to LOAD data.
-- SELECT is needed because UPDATE ... WHERE and INSERT ... ON CONFLICT read keys.
-- No DELETE, no right to create/drop tables; NEVER used by the dashboard.
CREATE ROLE pipeline_writer;
GRANT USAGE  ON SCHEMA public TO pipeline_writer;
GRANT SELECT, INSERT, UPDATE ON staging_customers, dim_customer, fact_ticket
  TO pipeline_writer;
CREATE POLICY pipeline_ghi ON fact_ticket FOR ALL TO pipeline_writer
  USING (true) WITH CHECK (true);

-- Principle: the dashboard connects with an account in the nha_phan_tich role,
-- NOT with the database owner account. This is checklist item 3.
```

Three things that must hold for the dashboard connection

1. The dashboard does NOT connect with the database owner account. If the dashboard has a vulnerability, the attacker gets full control. Use a separate account with SELECT only on the views it needs.

2. The dashboard reads the pseudonymised VIEWs, not the base tables. So even people with dashboard access cannot see customers' phone numbers and names.

3. Dashboard error messages must not print connection strings or table names. This is checklist item 5 and a very common mistake with Streamlit: set client.showErrorDetails = false in .streamlit/config.toml so the page does not print stack traces.

B4. Pipeline doc and Data dictionary — component 6 of Assignment 3

Pipeline doc — six sections

Section	Minimum content
1. Phase diagram	A diagram: source → extract → transform → quality → load → target. State which code file handles which phase.
2. Data sources	For each source: name, format, update frequency, owner, path or access method.
3. Targets and grain	For each target table: the grain in one sentence, the business key, the surrogate key.
4. Cleaning rules	A table of every rule: rule code, what it does, threshold, rows rejected in the latest run.
5. Schedule and backfill	Planned schedule; the command to backfill a failed day; what happens if the same day runs twice.
6. Known limitations and data latency	Source-data problems that cannot be fixed and which reports they affect; up to what point in time the figures are complete.

Data dictionary — a template with two rows filled in, each column with the six fields required by Assignment 3 (type · meaning · valid values · nullable · classification · security handling)

Table.column	Type	Meaning	Valid values	Nullable?	Classification	Security handling
dim_customer.sdt_chuan	VARCHAR(10)	Phone number normalised to 10 digits starting with 0. The business key used to merge duplicate customers.	Exactly 10 digits, starting with 0.	Yes (about 6.1% of source records lack a phone number)	PERSONALLY IDENTIFYING	Exists only in the base table; every analytical view uses the ma_khach_gia_danh column instead (HMAC-SHA256, computed in the pipeline). The nha_phan_tich role has no SELECT on this table.

Table.column	Type	Meaning	Valid values	Nullable?	Classification	Security handling
fact_ticket.tre_han	SMALLINT	0/1 flag: whether the ticket was closed after its due date. Stored as 0/1 so it can be summed; the late rate is AVG at query time.	Only 0 or 1.	No (NOT NULL)	INTERNAL	No handling needed. The nha_phan_tich role may read it.

B5. Self-check list [DA]

- ☐ Every warehouse column is classified into one of the four levels and recorded in the Data dictionary.
- ☐ Every personally identifying column in the analytical layer is pseudonymised or removed.
- ☐ Pseudonymisation uses HMAC with a secret key from an environment variable, computed in the pipeline; no bare hash, and the key is not stored in the database.
- ☐ Aggregate reports have a minimum group size, recorded with its rationale in the Data dictionary.
- ☐ There are at least two access roles with different privileges, and the dashboard uses the restricted role.
- ☐ The dashboard does not connect with the database owner account.
- ☐ Dashboard error messages do not print connection strings, table names or stack traces.
- ☐ No data file containing personal information is committed; sample data in the repo is pseudonymised.
- ☐ The Pipeline doc has all six sections, and the known-limitations section states a concrete business impact.
- ☐ The Data dictionary covers every column of every target table, each with the six fields shown in B4.
- ☐ Screenshot evidence: logging in with the nha_phan_tich role and running SELECT on dim_customer is refused (permission denied).

PART C — Track AI: Model card, bias report and responsible AI

C1. Model card — ten sections, component 6 of Assignment 3

A model card is the document that accompanies a model, written for the people who will USE it, not those who built it. The goal: readers can decide whether the model suits their job without reading the source code.

Section	Minimum content	Example: the Mekong Mobile issue-classification model
1. Basic information	Model name, version, date, author, contact	phan-loai-su-co v1.0 · 15/11/2026 · student <name>, student ID <number> · contact via the supervising lecturer
2. Intended use	The problem it solves, who uses it, in which process	SUGGESTS an issue category to intake staff while they type the issue description, to save data-entry time. The staff member makes the final decision.
3. Out-of-scope uses (should NOT be used for)	A concrete list of things this model is unsuitable for	NOT for automatically assigning technicians · NOT for deciding whether a ticket is covered by warranty · NOT for languages other than Vietnamese · NOT for evaluating staff performance
4. Training data	Source, sample size, time span, split, label distribution	4,000 issue descriptions from the internal system, 2024–2026 · stratified 70/15/15 split, seed 42 · labels: KHÁC 31%, MÀN_HÌNH 22%, PIN 16%, SẠC 13%, PHẦN_MỀM 9%, NƯỚC_VÀO 9%
5. Architecture and processing	Which model, which features, which preprocessing steps	TF-IDF 1–2 grams (min_df=2) + logistic regression with balanced class weights · preprocessing: lower-case, KEEP Vietnamese diacritics, replace phone numbers and serials with tags
6. Evaluation metrics	Which metrics, on which set, with measurement conditions and spread	Macro F1 on the test set = 0.750 (data v1.0, seed 42) · 5-fold cross-validation on the training set = 0.762 ± 0.031 · per-label F1: MÀN_HÌNH 0.83 · PIN 0.79 · SẠC 0.74 · PHẦN_MỀM 0.71 · NƯỚC_VÀO 0.69 (precision 0.86 · recall 0.57) · KHÁC 0.74
7. Baseline for comparison	The score of the simplest model	Majority-class guess: macro F1 = 0.079. The current model is a clear improvement over this floor.
8. Measured bias	How groups were formed, the gap between the best and worst group, a hypothesis for the cause	By service centre: gap 0.06 (acceptable) · by description length: descriptions under 10 words have F1 = 0.61 versus 0.81 for those over 25 words, a gap of 0.20 — above the threshold, see section 9
9. Risks and limitations	At least two concrete risks, each with a mitigation	(a) Very short descriptions give poor results: mitigate by raising the confidence threshold to 0.60 for descriptions under 10 words and handing over to manual selection. (b) The NƯỚC_VÀO label has 57% recall, and missing such a case can badly damage the device: mitigate by always showing NƯỚC_VÀO in the top 3 when the description contains water-related words.
10. Maintenance	When to retrain, signs of degradation, who is responsible	Retrain when the share of suggestions corrected by staff exceeds 35% in a month, or when a new issue category is added · monitor by logging each suggestion and the staff member's final choice

Three sections that decide the model-card mark

Section 3 (out-of-scope uses) is what separates a real model card from a technical description. It shows you understand the limits of what you built. A model card without it earns almost nothing.

Sections 8 and 9 (bias and risks) need NUMBERS and CONCRETE MEASURES. "The model may be biased" earns nothing; "a 0.20 gap between short and long descriptions, mitigated by a higher confidence threshold for short ones" earns marks.

Section 6 must state the measurement conditions. A number without a data version, a seed and the evaluation set cannot be verified — it breaks the course's third principle.

C2. Responsible-AI checklist — seven questions to answer yourself

The seven questions below are a shortened version, fitted to this course, of current responsible-AI frameworks. Answer all seven in writing, two to three lines each, and include them in Assignment 3 as part of the model card.

- ☐ 1. Who is affected by a WRONG prediction of this model, and what do they lose? Be specific; do not answer "the user".
- ☐ 2. Does that person know an AI system is involved in the decision? If not, why not, and should they be told?
- ☐ 3. Can that person object or ask for a result to be reviewed? If not, that is a limitation you must state.
- ☐ 4. Does the model perform worse for some group of people or some group of data? Have you MEASURED it, or are you guessing?
- ☐ 5. Does the model use any feature whose use is ethically inappropriate (gender, age, place of residence, when these are irrelevant to the problem)?
- ☐ 6. Was the training data collected lawfully, and do the people who provided it know it is used for this purpose?
- ☐ 7. If the model stops working, does the system still run, and who notices that it has stopped?

Questions 4 and 5 are the ones most often answered superficially

Question 4 — "the model may be biased" is not an answer. A proper answer names the grouping, the gap in numbers and a conclusion. If you have not measured it, say honestly "I have not measured along this dimension" — honesty earns marks, guessing does not.

Question 5 — many models use indirect features without realising it. Example: using the province to predict warranty eligibility can turn into regional discrimination. Go through each feature and ask "if users knew they were judged on this feature, would they find it reasonable?".

Link to the exam: question 4b of the final exam asks about a legal or ethical responsibility when releasing a model to the public. These seven questions are your source for answering it.

C3. Three transparency obligations to know — updated to October 2026

The content below is at AWARENESS level, so you know where to look when you build a real product. It is not legal advice and does not replace checking the current texts. Rules in this area change fast — verify them at the official source before relying on them.

Obligation	What it says (awareness level)	How it applies to your prototype
Tell users they are interacting with AI	Under Article 50 of the EU AI Act (Regulation (EU) 2024/1689), a system that interacts directly with people (such as a chatbot) must inform them that they are interacting with AI. This applies from 2 August 2026; the Digital Omnibus amendment package on AI did not postpone it. Vietnam's Law on Artificial Intelligence (in force since 1 March 2026) likewise requires that users can recognise they are interacting with an AI system.	If your product has a chatbot or automatic replies, add a clear notice. If it is only an internal suggestion model for staff, state in the interface that this is an automatic suggestion.
Mark AI-generated content	AI-generated content (text, images, audio, video) must be marked in a machine-readable way; deepfakes must be clearly disclosed. Under the Digital Omnibus, generative systems placed on the market BEFORE 2 August 2026 have a transition period until 2 December 2026; new systems must comply immediately.	If your prototype generates content, add an identifying mark. If it generates none, write "not applicable" with a reason.
State the limits of use	Whoever releases a model is responsible for stating what it is for and what it is NOT for, so that others do not misuse it.	This is exactly section 3 of the model card. It applies to EVERY AI prototype, internal ones included.

Important note on legal scope

The EU AI Act is a European Union regulation. Your product may fall under it if it serves users in the EU.

In Vietnam: the Law on Personal Data Protection (No. 91/2025/QH15, in force since 1 January 2026) and the Law on Artificial Intelligence (passed 10 December 2025, in force since 1 March 2026, classifying AI systems into three risk levels: high · medium · low). When you build a real product, check the current texts and their implementing decrees.

What this course asks of you is not to memorise laws, but to recognise when a situation is problematic, know that rules exist for it, and know where to look.

Official sources: digital-strategy.ec.europa.eu (EU AI Act) · vanban.chinhphu.vn (Vietnamese legal documents). The dates in this guide are correct as of October 2026 and must be re-checked every semester.

C4. Securing the model service — four items specific to track AI

Item	Risk	What to do for Assignment 3
Limit input size	A description several megabytes long makes the model service hang or run out of memory	Set a maximum length in the input validation schema (e.g. 2,000 characters) and return 422 when exceeded
Limit call rate	Someone calling continuously exhausts the model service and the core business flow suffers	At prototype level: set a maximum timeout on calls and ensure graceful degradation. Full rate limiting belongs to Graduation Topic 2

Item	Risk	What to do for Assignment 3
Do not log user data	Issue descriptions may contain phone numbers, customer names, serials — logging them leaks personal data	Logs record only description length, predicted label, confidence and processing time. Do NOT log the description text
Do not reveal model details in error messages	Stack traces reveal file paths, library versions, model structure	Catch every exception at the outer layer and return a generic message; details go only to the server-side log

C5. Self-check list [AI]

- ☐ The model card has all ten sections in table C1.
- ☐ Section 3 (out-of-scope uses) lists at least three concrete items.
- ☐ Section 6 (metrics) states the measurement conditions: data version, seed, evaluation set, spread.
- ☐ Section 8 (bias) has real measured figures for at least two groupings.
- ☐ Section 9 (risks) has at least two risks, each with a concrete, feasible mitigation.
- ☐ All seven responsible-AI questions are answered, two to three lines each.
- ☐ Question 4 is answered honestly — any dimension not measured is stated as not measured.
- ☐ The model service limits input length and returns 422 when exceeded.
- ☐ The model service log does NOT record the user's description text.
- ☐ Model service error messages do not reveal file paths or library versions.
- ☐ If the prototype interacts directly with people, it shows a notice that this is AI.

Quick comparison of the three tracks — a cross-check table

Security checklist item	Track SE	Track DA	Track AI
1. No secret in the repo	JWT_SECRET, connection strings and API keys all in .env	Warehouse connection string and pseudonymisation key (PSEUDO_KEY) in .env	API keys for external services (if any) in .env
2. Input validation	Validator in the application layer; three anti-injection grep commands	Check data quality before loading; do not load below the threshold	Input schema of the prediction service; length limit
3. Access control	Owner check in the service layer; a test case per OWASP A01	Database roles; pseudonymised views; the dashboard uses a restricted role	The model service is reachable only from the compose internal network
4. Hashing or encryption	Passwords hashed with bcrypt or argon2	Personally identifying columns pseudonymised with keyed HMAC	Not directly applicable; state the reason
5. Error messages reveal no internals	Map business codes to HTTP codes; never return stack traces	Dashboard prints no connection string; Streamlit sets client.showErrorDetails = false	No model file paths or library versions revealed
6. Safe logging	A function masks password and token fields before logging	Pipeline logs do not record the contents of personal-data rows	Logs record only length, label, confidence — never the text
7. Pinned dependencies	package-lock.json committed; npm audit at a basic level	requirements.txt uses ==; pip-audit at a basic level	requirements.txt uses ==; library versions must match those used when the model was saved
Technical documentation (component 6)	API docs: every endpoint with seven sections and REAL examples	Six-section Pipeline doc + Data dictionary with six fields per column	Ten-section model card + the seven responsible-AI answers

Three things to do this week, before the Session 16 workshop (Assignment 3 submission)

1. Scan for secrets first — including the Git history, because the repo is PUBLIC. Run git log --all -- oneline -- .env (it must print NOTHING) and git log -p --all | grep -inE "password|secret|api_key|token" | head (fine on a small repo). If you find anything, rotate every affected secret immediately and record that in the documentation. This is checklist item 1 and the most serious one.

2. Fill in the seven-item checklist by ACTUALLY RUNNING checks, not from memory. Each item needs concrete evidence: a command you ran with **a screenshot of the result**, or a file path. An honest "not done" costs no marks.

3. Write your track's technical documentation IN PARALLEL with the security checks, not at the end. Many parts of the documentation (possible error codes, known limitations, limits of use) are direct results of the security checks — doing them together is much faster.